

Mobil Geliştiriciler İçin Networking

Ömer Faruk Yavuz

November 2025

Uzak İki Ofisi Nasıl Bağlarız?

Uzak iki ofisin birbirine güvenli ve stabil biçimde bağlanması için genellikle iki teknoloji kullanılır:

a) VPN (Virtual Private Network)

VPN, iki uzak lokasyon arasında **şifreli bir tünel** oluşturur.

- IPsec veya SSL tabanlı olabilir.
- Kurumsal ortamlarda site-to-site VPN kullanılır.
- İnternet hattı üzerinden kurulduğu için maliyet düşüktür.

b) Cloud Interconnect / Private Link

Bulut sağlayıcıların sunduğu özel hatlardır. Genellikle şu isimlerle bilinir:

- AWS Direct Connect
- Azure ExpressRoute
- Google Cloud Interconnect

Bu bağlantılar:

- İnternete çıkmadan, **özel bir WAN hattı** sağlar,
- Daha düşük latency sunar,
- SLA garantisi olduğu için kurumsal yapılarda tercih edilir.

Internetworking Nedir?

Internetworking, farklı ve bağımsız ağların (router veya gateway gibi ara cihazlar kullanılarak) **birbirine bağlanması** ve tek bir büyük ağ gibi çalışmasıdır.

Bu kavramın temel noktaları şunlardır:

a) Tanım

Internetworking, “inter” (arasında) ve “networking” (ağ kurma) kelimelerinden oluşur. Amaç, farklı teknolojilere, farklı IP aralıklarına veya farklı topolojilere sahip ağları **birleştirip iletişim kurabilir hale getirmektir**.

b) Nasıl sağlanır?

Bu birleşme genellikle şu cihazlarla gerçekleştirilir:

- **Router** (IP seviyesinde yönlendirme yapar)
- **Gateway** (farklı protokol veya mimarileri bağlar)
- **Layer-3 Switch** (gelişmiş yönlendirme yapabilir)

c) Internetwork’ün Özelliği

Bağlanan ağlar şu türlerde olabilir:

- Özel kurum ağı (private network)
- Kamu ağı
- Endüstriyel ağ
- Ticari ağ

Tüm bu ağlar **intermediate devices** üzerinden bağlanınca, tek bir büyük “internetwork” oluştururlar.

d) İnternet = En büyük internetwork

Bugünkü küresel internet; milyonlarca bağımsız LAN, WAN ve ISP ağının router’lar üzerinden birbirine bağlanmasıyla oluşmuş dev bir internetwork yapısıdır.

OSI Modelindeki User Support (Software) Layers Hangileridir?

OSI (Open Systems Interconnection) modelinde uygulamaya en yakın olan, son kullanıcıya hizmet veren katmanlara **User Support Layers** veya **Software Layers** denir. Bu katmanlar, uygulamaların ağ üzerinde veri alışverişi yapmasını sağlar ve genellikle **iş mantığı, veri formatlama ve oturum yönetimi** ile ilgilenir.

User Support Layers (Layer 5–7)

1. **Application Layer (Katman 7)**
2. **Presentation Layer (Katman 6)**
3. **Session Layer (Katman 5)**

a) Application Layer (Layer 7)

Son kullanıcıya en yakın katmandır. Uygulama protokollerinin çalıştığı katmandır.

Görevleri:

- HTTP, HTTPS, FTP, SMTP, DNS gibi protokolleri çalıştırır
- Kullanıcı ile ağ arasındaki etkileşimi sağlar

Gerçek Hayat Örnekleri:

- Tarayıcıda bir web sitesini açmak (HTTPS)
- Gmail üzerinden e-posta göndermek (SMTP)
- Spotify API üzerinden şarkı listesi çekmek (REST/HTTPS)
- DNS sorgusu yapmak (DNS protokolü)

b) Presentation Layer (Layer 6)

Verinin **formatlanması, sıkıştırılması ve şifrelenmesinden** sorumludur.

Görevleri:

- Veri şifreleme (Encryption)
- Veri sıkıştırma (Compression)
- Veri format çevirimi (JSON, XML, JPEG, MP4 vb.)

Gerçek Hayat Örnekleri:

- HTTPS bağlantısındaki TLS işlemleri
- Bir görüntünün JPEG formatına encode edilmesi
- Metnin UTF-8 formatına dönüştürülmesi
- Videonun H.264 olarak sıkıştırılması

c) Session Layer (Layer 5)

İki uygulama arasındaki oturumu (session) yönetir.

Görevleri:

- Oturum açma/kapama
- Oturumu yeniden başlatma
- Checkpoint mekanizmaları

Gerçek Hayat Örnekleri:

- SSH oturumunun kurulması ve sürdürülmesi
- Netflix'te bağlantı kopunca session'ın yeniden kurulması
- Zoom/Teams video görüşmesindeki oturum yönetimi
- gRPC ve RPC çağrılarındaki session handshake işlemleri

OSI Modelindeki Hardware Layers (Network Support Layers) Hangileridir?

OSI modelinin alt üç katmanı, ağın fiziksel olarak çalışmasını sağladıkları için **Hardware Layers** veya **Network Support Layers** olarak adlandırılır. Bu katmanlar, bit düzeyinden çerçevelere, yönlendirmeden fiziksel sinyale kadar tüm düşük seviyeli ağ faaliyetlerini yönetir.

Network Support Layers (Layer 1–3)

1. **Physical Layer (Katman 1)**
2. **Data Link Layer (Katman 2)**
3. **Network Layer (Katman 3)**

1. Physical Layer (Layer 1)

Bu katman veriyi **bit düzeyinde** taşır ve fiziksel sinyal iletimini sağlar.

Görevleri:

- Voltaj seviyeleri, ışık sinyalleri, radyo sinyalleri
- Kablolama (Fiber, UTP), frekanslar, bant genişliği
- Bitlerin iletilmesi (0 ve 1)

Gerçek Hayat Örnekleri:

- Ethernet kabloları (Cat5e, Cat6)
- Fiber optik kablolar
- Wi-Fi radyo sinyalleri
- Donanım: Hub, Repeater

2. Layer 2 (Data Link Layer)

Layer 2, aynı yerel ağ (LAN) içindeki cihazlar arasındaki iletişimi sağlayan katmandır. Bu katman, **MAC adreslerini** kullanarak çerçevelerin (frames) doğru cihaza yönlendirilmesini sağlar.

Ana Özellikler:

- Kimlik: **MAC adresi**
- Veri birimi: **Frame**
- Temel cihaz: **Switch**

- Kapsam: Aynı ağ (LAN)

Gerçek Hayat Örnekleri:

- Bir switch'in MAC adres tablosu ile trafiği doğru porta iletmesi
- Bilgisayarın kendi ağı içindeki başka bir cihaza veri göndermesi
- Ethernet çerçevelerinin taşınması

Benzetme:

Aynı apartmandaki komşuya kapı numarasını kullanarak mektup bırakmak.

3. Layer 3 (Network Layer)

Layer 3, farklı ağlar (LAN → WAN → Internet) arasında paketlerin yönlendirilmesini sağlar. Bu katman **IP adresleri** ile çalışır ve en kısa/uygun rotayı belirler.

Ana Özellikler:

- Kimlik: **IP adresi**
- Veri birimi: **Packet**
- Temel cihaz: **Router**
- Kapsam: Ağlar arası iletişim (internetworking)

Gerçek Hayat Örnekleri:

- Bir router'ın farklı ağlara paket yönlendirmesi
- Bir cihazın internete (örneğin 8.8.8.8'e) erişmesi
- OSPF veya BGP ile rota hesaplaması yapılması

Benzetme:

Başka bir şehre giden otobüsün rotasını seçmek.

Özet

Network Support (Hardware) katmanları:

- **Physical Layer (1)** → bitler, sinyal, kablo, radyo
- **Data Link Layer (2)** → çerçeve, MAC adresi, switch
- **Network Layer (3)** → IP adresi, router, yönlendirme

Bu üç katman, ağın fiziksel olarak çalışmasını ve paketlerin iki cihazdan tüm internete kadar taşınmasını sağlar.

HTTPS Protokolü Nedir?

HTTPS (HyperText Transfer Protocol Secure), HTTP protokolünün şifrelenmiş ve güvenli versiyonudur. İstemci (client) ile sunucu (server) arasındaki veri alışverişini, **TLS/SSL** kullanarak şifreler ve böylece üçüncü tarafların veriyi okumasını veya değiştirmesini engeller.

Temel Özellikler

- Varsayılan port numarası: **443**
- HTTP + TLS/SSL birleşiminden oluşur.
- Veri gizliliği, bütünlüğü ve kimlik doğrulaması sağlar.

HTTPS'in Sağladığı Üç Temel Güvenlik Özelliği

1. **Gizlilik (Confidentiality)** Tüm trafik TLS ile şifrelenir. Araya giren biri (MITM) veriyi okuyamaz.
2. **Bütünlük (Integrity)** Veri gönderildiği gibi ulaşır. TLS, verinin değiştirilmesini engeller.
3. **Kimlik Doğrulama (Authentication)** Sunucunun gerçekten iddia ettiği sunucu olduğunu kanıtlar. (Örnek: `google.com` gerçekten Google'ın sertifikasına sahiptir.)

HTTPS Nasıl Çalışır? (Özet Akış)

1. Client, sunucuya bağlanır: `https://example.com`
2. Sunucu, TLS sertifikasını gönderir.
3. Client sertifikanın doğruluğunu CA üzerinden kontrol eder.
4. Client ve sunucu bir **ortak oturum anahtarı** üretir.
5. Sonraki tüm iletişim bu anahtar ile **şifrelenir**.

Gerçek Hayat Örnekleri

- Bir bankanın sitesine giriş yaparken (internet bankacılığı)
- E-ticaret ödemelerinde (kredi kartı işlemleri)
- WhatsApp Web gibi uygulamalarda web trafiğinin şifrelenmesi
- API isteklerinin güvenli taşınması (REST/GraphQL)

Özet

HTTPS = HTTP + TLS Port = 443 Amaç = Güvenli, şifreli, doğrulanmış iletişim.

Application Layer'ın Sağladığı Servisler Nelerdir?

Application Layer (OSI Katman 7), son kullanıcıya en yakın katmandır ve kullanıcı uygulamalarının ağ üzerinden iletişim kurmasını sağlayan yüksek seviye servisleri sunar. Bu katman, doğrudan uygulama protokollerini çalıştırır ve veri alışverişini üst seviyede yönetir.

Application Layer Servisleri

1. Mail Services

- E-posta gönderme (SMTP)
- E-posta alma (POP3, IMAP)

2. Directory Services

- DNS (Alan adlarının IP adresine çevrilmesi)
- LDAP (Kullanıcı, grup, rol yönetimi)

3. File Transfer Services

- FTP (File Transfer Protocol)
- SFTP / FTPS (Güvenli dosya transferi)

4. Access Management

- Kullanıcı erişim kontrol mekanizmaları
- Kimlik doğrulama protokolleri (OAuth, OpenID Connect)

5. Network Virtual Terminal

- Uzak terminal oturumu sağlama (Telnet)
- Uzak komut çalıştırma

Bu Servislerin Görevleri

- Uygulamaların network üzerinden haberleşmesini sağlamak
- Yüksek seviye protokolleri çalıştırmak
- Sunucu-istemci arasındaki veri alışverişini yönetmek

Header ve Trailer Hangi OSI Katmanlarında Eklenir?

OSI modelinde veri üst katmanlardan alt katmanlara doğru ilerlerken, her katman kendi kontrol bilgilerini verinin başına (**header**) veya sonuna (**trailer**) ekler. Bu işleme **encapsulation** (kapsülleme) denir.

Header Ekleme

Aşağıdaki katmanlar, veriye **sadece header** ekler:

- **Application Layer (7)**
- **Presentation Layer (6)**
- **Session Layer (5)**
- **Transport Layer (4)**
(TCP/UDP header ekler)
- **Network Layer (3)**
(IP header ekler)

Her header katmana özgü kontrol bilgilerinden oluşur:

- TCP Port bilgileri (Katman 4)
- IP adresleri (Katman 3)
- Protokol tip bilgisi (Katman 3-4)

Trailer Ekleme

Sadece Data Link Layer (Katman 2) trailer ekler.

- Ethernet Frame Trailer (FCS/CRC)
- Hata tespit bilgisi içerir

Özet

- **Header** → Katman 7, 6, 5, 4, 3 tarafından eklenir.
- **Trailer** → Sadece **Data Link Layer (Katman 2)** ekler.

Encapsulation Görselleştirme

Data
↓
[Layer 7 Header] Data
↓
[Layer 6 Header] [Layer 7 Header] Data
↓
[Layer 5 Header] [Layer 6 Header] [Layer 7 Header] Data
↓
[Transport Header] [Layer 5 Header] [Layer 6 Header] [Layer 7 Header] Data
↓
[IP Header] [Transport Header] [Layer 5 Header] [Layer 6 Header] [Layer 7 Header] Data
↓
[Ethernet H] [IP H] [Transport H] [Layer 5 H] [Layer 6 H] [Layer 7 H] Data [Ethernet Trail]

Veri Alt Katmanlardan Üst Katmanlara Doğru İlerlerken Ne Olur?

OSI modelinde bir veri paketi **alt katmanlardan üst katmanlara** doğru çıktığında, her katman kendi eklediği kontrol bilgisini (başlık/header ve varsa trailer) **çıkartır**. Bu sürece **decapsulation** (kapsülü açma) denir.

Temel Mantık

Alt katmanlardan gelen paket → her katmanda o katmanın header'ı çıkarılır → üst katmana sadece ihtiyaç duyulan veri aktarılır.

Katman Bazında Decapsulation Süreci

1. **Layer 1 (Physical)**: Bitleri alır ve Layer 2'ye iletir.
2. **Layer 2 (Data Link)**: Ethernet Header + Trailer (FCS/CRC) çıkarılır.
3. **Layer 3 (Network)**: IP Header çıkarılır.
4. **Layer 4 (Transport)**: TCP/UDP Header çıkarılır (port bilgisi bu katmanda okunur).
5. **Layer 5–7 (Session/Presentation/Application)**: Uygulama verisi kademeli olarak soyutlanır.

Örnek Akış

Bir HTTP isteği alındığında şu işlem gerçekleşir:

```
[Ethernet Header] [IP Header] [TCP Header] [HTTP Data] [Ethernet Trailer]
↓ (L2 decapsulation)
[IP Header] [TCP Header] [HTTP Data]
↓ (L3 decapsulation)
[TCP Header] [HTTP Data]
↓ (L4 decapsulation)
[HTTP Data]
↓ (L5--L7)
Application Layer'a aktarılır
```

Özet

- Veri yukarı çıktıkça **header'lar çıkarılır**.
- Sadece uygulama seviyesindeki veri (payload) kalır.
- Bu sürece **decapsulation** denir.

Veri Üst Katmanlardan Alt Katmanlara Doğru İlerlerken Ne Olur?

OSI modelinde bir veri paketi **üst katmanlardan alt katmanlara** doğru ilerlediğinde, her katman kendi kontrol bilgisini verinin başına ekler. Bu işleme **encapsulation** (kapsülleme) denir.

Üst katmandaki uygulama verisi, alt katmanlara indikçe yeni header'lar eklenir ve sonunda fiziksel sinyale dönüştürülüp iletilir.

Kapsülleme (Encapsulation) Adımları

1. **Layer 7 (Application)**: Uygulama verisi üretilir (ör. HTTP Request).
2. **Layer 6 (Presentation)**: Veri şifrelenir, sıkıştırılır veya encode edilir.
3. **Layer 5 (Session)**: Oturum bilgileri hazırlanır.
4. **Layer 4 (Transport)**: TCP/UDP header eklenir (port numaraları burada).
5. **Layer 3 (Network)**: IP header eklenir (kaynak hedef IP).
6. **Layer 2 (Data Link)**: Ethernet header + trailer eklenir (MAC adresleri, FCS).
7. **Layer 1 (Physical)**: Veri elektriksel/optik/radyo sinyallerine dönüştürülür.

Encapsulation Örneği

Bir HTTP isteği gönderilirken oluşan yapı şöyledir:

```
Application Data (HTTP)
  ↓ L6
[Presentation Header] HTTP Data
  ↓ L5
[Session Header] ...
  ↓ L4
[TCP Header] [HTTP Data]
  ↓ L3
[IP Header] [TCP Header] [HTTP Data]
  ↓ L2
[Ethernet Header] [IP Header] [TCP Header] [HTTP Data] [Ethernet Trailer]
  ↓ L1
Sinyal olarak iletilir
```

Eklenen Bilgiler Neleri İçerir?

- L4 (Transport): Port numaraları, sequence/ack numaraları
- L3 (Network): IP adresleri
- L2 (Data Link): MAC adresleri, FCS/CRC hata tespiti

Özet

Üst katmandan alt katmana indikçe **header eklenir**. Katman ne kadar aşağıdaysa eklenen bilgi o kadar "donanıma yakın" yapıdadır.

Mobil Uygulamalarda DNS (Domain Name System)

DNS, bir alan adını (`api.example.com`) ilgili IP adresine çözen sistemdir. Mobil uygulamalarda DNS çözümlemeleri performans ve güvenilirlik açısından kritik öneme sahiptir.

Neden Mobilde DNS Daha Kritik?

- Mobil ağlarda DNS cache süresi çok kısa olabilir.
- Ağ geçişlerinde (Wi-Fi ↔ LTE) DNS cache boşa düşer.
- Captive portallar (otel, havaalanı Wi-Fi) DNS'i **redirect** edebilir.
- İlk API çağrısının **300–800 ms** gecikmesinin ana sebebi DNS'tir.
- DNS çözümlemesi başarısız olursa tüm uygulama **offline** görünür.

DNS Lookup Süreci

Bir mobil cihaz bir alan adına istek yaptığında sırasıyla şu adımlar gerçekleşir:

1. Cihaz DNS cache'ine bakar.
2. Yoksa işletim sistemi DNS resolver'a sorar.
3. Resolver, ISP veya işletim sistemi DNS sunucusuna istek gönderir.
4. IP adresi çözülür ve uygulamaya döndürülür.

Mobil Cihazlarda DNS Davranışları

Android:

- Uygulamaya özel DNS cache yoktur.
- OS genelde kısa süreli (1 dakika) DNS cache tutar.

iOS:

- DNS cache süresi daha agresif olarak temizlenir.
- Ağ değişimlerinde tüm cache **invalidate** olur.

DNS Kaynaklı Mobil Problemler

- **Cold-start latency** artar.
- Retrofit/Alamofire gibi HTTP client'lar ilk istekte **timeout** atabilir.
- Captive ağlarda DNS sizi login sayfasına yönlendirir.
- Yanlış DNS konfigürasyonunda API çöker gibi görünür.

Mobil DNS Performansı: Basit Açıklama

Bu bölümde DNS gecikmesi, pre-connect, hızlı DNS sağlayıcıları, TTL ve CDN gibi kavramlar, mobil uygulama geliştirme açısından **en basit haliyle** açıklanmaktadır.

1. Pre-Connect Nedir?

Pre-connect, uygulama açılır açılmaz API sunucusuna **erken bağlantı** kurulması anlamına gelir.

Uygulama açılır açılmaz, kullanıcı henüz hiçbir işlem yapmadan: DNS çözülür, TCP bağlantısı kurulur ve TLS el sıkışması tamamlanır.

Böylece ilk API isteği yapıldığında:

- DNS beklemez
- TCP bağlantısı kurulmasını beklemez
- TLS işlemlerini beklemez

Sonuç: **İlk API isteği anında gider.**

2. Hızlı DNS (Cloudflare / Google DNS) Neden Önemlidir?

Bazı mobil operatörlerin DNS sunucuları çok yavaştır. Örneğin:

- Yavaş bir DNS: 400–600 ms
- Hızlı DNS (Cloudflare/Google): 10–30 ms

Bu fark, uygulamanın açılış hızını doğrudan etkiler.

Hızlı DNS kullanmak = Uygulamanın daha hızlı açılması.

3. TTL Nedir ve Neden Stabil IP Kullanmak Önemlidir?

TTL, “Bu DNS bilgisi ne kadar süre saklansın?” sorusunun cevabıdır.

- TTL kısa olursa: DNS çok sık yeniden çözülür.
- TTL uzun olursa: DNS daha az çözülür, performans artar.

Mobil cihaz Wi-Fi ile LTE arasında geçiş yaptığında DNS cache silinebilir. Bu durumda yeniden DNS çözümü gerekir ve gecikme oluşur.

API sunucuları stabil IP kullanırsa:

- TTL daha uzun tutulabilir
- DNS daha seyrek çözülür
- Daha az gecikme ve hata oluşur

4. CDN DNS'i Nasıl Hızlandırır?

CDN (Content Delivery Network), DNS çözümünü **kullanıcıya en yakın noktadan** yapar.

- Normal DNS: Avrupa'daki sunucudan cevap (80–120 ms)
- CDN DNS: İstanbul'daki edge noktadan cevap (5–10 ms)

CDN = DNS cevabını sana en yakın yerden veren sistem.

Bu nedenle CDN kullanımı DNS gecikmesini önemli ölçüde azaltır.

Özet

- **Pre-connect**: API'a giden yolu önceden hazırlamak.
- **Hızlı DNS**: Uygulamanın daha hızlı açılmasını sağlar.
- **TTL**: DNS bilgisinin ne kadar saklanacağını belirler.
- **CDN**: DNS sorgularını en yakın noktadan cevaplayarak hız kazandırır.

Mobil uygulamalardaki ilk yavaşlık çoğu zaman DNS çözümleme süresinden kaynaklanır. Bu teknikler, açılış süresini belirgin şekilde hızlandırır.

Mobil Uygulamalarda HTTPS, TLS ve Certificate Pinning

Mobil uygulamalar ile sunucu arasındaki tüm iletişim, güvenlik nedeniyle **HTTPS** üzerinden yapılır. HTTPS, HTTP protokolünün **TLS** (Transport Layer Security) ile şifrelenmiş halidir.

1. HTTPS Nedir?

- Varsayılan portu: **443**
- HTTP + TLS birleşiminden oluşur.
- Amaç: veriyi şifrelemek ve güvenli iletişim sağlamak.

HTTPS = HTTP'nin şifrelenmiş hali.

2. TLS (Transport Layer Security) Nedir?

TLS, istemci ile sunucu arasındaki iletişimi şifreleyen protokoldür. Mobil uygulamalarda TLS üç önemli güvenlik özelliği sağlar:

- **Gizlilik** (veri şifrelenir)
- **Bütünlük** (veri değiştirilmez)
- **Kimlik doğrulama** (sunucu gerçekten o sunucu mu?)

3. TLS Neden Mobilde Yavaşlık Yaratabilir?

Bir HTTPS isteği yapılmadan önce şu üç adım gerçekleşir:

1. DNS çözümü
2. TCP bağlantısı kurulması
3. **TLS handshake** (sertifika doğrulama)

Bu işlem özellikle mobilde:

- düşük sinyal gücünde,
- baz istasyonu değişimlerinde (handover),
- Wi-Fi ↔ LTE geçişlerinde

100–500 ms ek gecikme yaratabilir.

4. Certificate Pinning Nedir?

Certificate Pinning, istemcinin sadece **belirli bir sertifikaya** güvenmesini sağlayan bir güvenlik tekniğidir.

Pinning: “Bu sunucu sadece şu sertifikaya sahip olursa bağlan.” demektir.

Amaç: Man-in-the-Middle (MITM) saldırılarını önlemek.

Pinning Nerelerde Kullanılır?

- Bankacılık uygulamaları
- Ödeme sistemleri
- Hassas veriye sahip sağlık uygulamaları

5. Pinning Mobilde Nasıl Yapılır?

Android (OkHttp):

```
CertificatePinner pinner = new CertificatePinner.Builder()
    .add("api.example.com", "sha256/...")
    .build();
```

iOS (NSURLSession Delegate):

```
func urlSession(...) {
    // serverTrust doğrulaması
}
```

6. Pinning’in Dezavantajları

- Sertifika değiştiğinde uygulama güncellemesi gerekir.
- CI/CD pipeline ile otomatik sertifika güncelleme önemlidir.

Özet

- HTTPS = Şifreli iletişim.
- TLS = Güvenli bağlantının temel protokolü.
- Handshake = İlk isteğin gecikme kaynağı.
- Pinning = “Bu sertifika yoksa bağlanma.” kuralı.

Mobil uygulamalarda tüm API iletişimi HTTPS üzerinden yapılmalı, güvenlik kritik durumlarda Certificate Pinning uygulanmalıdır.

Latency, RTT, Packet Loss ve Jitter

Mobil cihazların ağ performansı; baz istasyonu geçişleri, zayıf sinyal, Wi-Fi ve LTE arasındaki geçişler gibi sebeplerle sürekli değişir. Bu bölümde mobil uygulamaların performansını doğrudan etkileyen dört kritik kavram sade şekilde açıklanmıştır.

1. Latency (Gecikme)

Latency, bir isteğin sunucuya gidip cevap almasının ne kadar sürdüğünü gösterir. Mobil ağlarda latency genellikle **30 ms – 300 ms** arasındadır.

- Wi-Fi: 20–50 ms
- LTE/4G: 50–150 ms
- 3G: 150–300 ms
- Zayıf sinyal: 300+ ms

Latency ne kadar yüksekse API çağrısı o kadar yavaş hissedilir.

2. RTT (Round-Trip Time)

RTT, bir paketin sunucuya gidip geri dönme süresidir. Latency ile karıştırılmaması gerekir:

Latency = tek yön gecikme
RTT = gidiş + dönüş gecikmesi

TLS handshake gibi işlemler multiple RTT gerektirdiği için mobilde açılış süreleri uzayabilir.

3. Packet Loss (Paket Kaybı)

Packet loss, gönderilen veri paketlerinin bir kısmının hedefe ulaşmamasıdır.

Mobil ağlarda yaygın sebepler:

- Zayıf LTE sinyali
- Baz istasyonu değişimi (cell handover)
- Wi-Fi parazitleri
- Aşırı yüklenmiş ağ

Etkisi:

- TCP bağlantılarında yeniden gönderim (retransmission) olur.
- Bu durum API çağrılarını birkaç saniye geciktirebilir.
- Video call'da kalite düşer, ses kesilir.

4. Jitter (Gecikme Oynaklığı)

Jitter, paketlerin farklı zamanlarda varmasıdır. Yani gecikme sabit değildir; dalgalanır.

Bir paket 40 ms, diğeri 120 ms, sonra 80 ms gelir → işte bu jitter'dır.

Jitter özellikle şuralarda kritiktir:

- WebRTC voice/video
- Online gaming
- Live streaming

Etkileri:

- Sesin bozulması
- Görüntünün donması
- Video/ses senkron kaybı

Özet

- **Latency:** Tek yön gecikme.
- **RTT:** Gidiş-dönüş süresi.
- **Packet loss:** Paketlerin kaybolması.
- **Jitter:** Paketlerin gecikmesinin dalgalanması.

Mobil ağlar sabit değildir; kullanıcı hareket ettikçe sinyal güçleri, baz istasyonları ve ağ kalitesi değişir. Bu yüzden API performansı ve gerçek zamanlı iletişim bu dört kavramdan güçlü şekilde etkilenir.

Mobil Ağlarda Timeout, Retry ve Backoff Stratejileri

Mobil ağlar sabit değildir; kullanıcı hareket ederken sinyal gücü, baz istasyonu, Wi-Fi erişim noktası veya IP adresi değişebilir. Bu nedenle API isteklerinde zaman zaman gecikme, kopma veya geçici hatalar oluşur. Bu durumları yönetmek için **timeout**, **retry** ve **backoff** stratejileri zorunludur.

1. Timeout Nedir?

Timeout, bir isteğin belirlenen süre içinde yanıt alamazsa **hata ile sonlandırılmasıdır**. Mobilde timeout'lar genellikle şu durumlarda tetiklenir:

- Zayıf LTE/Wi-Fi sinyali
- Ağ geçişi (Wi-Fi ↔ LTE)
- Paket kaybı (packet loss)
- DNS çözümlemesinin uzun sürmesi

Uygulama, sonsuza kadar beklememek için **kısa ve makul timeout değerlerine** sahip olmalıdır.

2. Retry (Tekrar Deneme)

Retry, başarısız olan isteğin belirli durumlarda **yeniden gönderilmesi**dir. Ancak retry yalnızca şu tür hatalar için yapılmalıdır:

- Geçici ağ hataları (timeout, connection lost)
- DNS geçici hataları
- 502 / 503 / 504 sunucu hataları

Aşağıdaki durumlarda retry yapılmamalıdır:

- 400 (bad request)
- 401 (unauthorized)
- 403 (forbidden)
- İşlem idempotent değilse (örneğin bir ödeme yapmak)

3. Backoff Stratejisi

Retry işlemleri arasında gecikme eklenmesine **backoff** denir. Bu, sunucuyu ve ağı aşırı yüklemekten korur.

En sık kullanılan yöntem:

Exponential Backoff: Her retry denemesinde bekleme süresi katlanarak artar.

Örnek:

1. deneme → hemen
2. deneme → 1 saniye sonra
3. deneme → 2 saniye sonra
4. deneme → 4 saniye sonra

Bu strateji özellikle mobilde önemlidir çünkü:

- Kullanıcı tünelden geçiyor olabilir (sinyal düşer)
- IP değişmiş olabilir
- Baz istasyonu değişmiş olabilir
- Wi-Fi internet bağlantısı kesilmiş olabilir

4. Idempotent İsteklerde Retry

Idempotent istekler tekrarlandığında sonuç değişmez.

Örneğin:

- GET /profile
- PUT /user/settings

Bu isteklerde retry güvenlidir.

Ancak idempotent olmayan isteklerde retry sakıncalıdır:

- POST /payment
- POST /order

Bu tür istekler **aynı işlemi iki kez** gerçekleştirebilir.

5. Mobilde Doğru Timeout ve Retry Örnek Değerleri

Android (OkHttp) önerilen değerler:

- connectTimeout: 5 saniye
- readTimeout: 10 saniye
- writeTimeout: 10 saniye
- retryOnConnectionFailure: true

iOS (NSURLSession) önerilen değerler:

- timeoutIntervalForRequest: 5–10 saniye
- timeoutIntervalForResource: 20–30 saniye

Özet

- **Timeout:** İstek çok uzun sürerse hata ver.
- **Retry:** Geçici ağ hatalarında isteği yeniden gönder.
- **Backoff:** Her retry arasında gecikmeyi artır.
- **Idempotent isteklerde retry güvenli,** diğerlerinde dikkat gerektirir.

Mobil cihazlarda ağ sürekli değiştiği için doğru timeout, retry ve backoff stratejileri uygulama performansını ve kararlılığını belirgin şekilde artırır.

DNS ve IP Adresi Nedir?

1. DNS (Domain Name System)

DNS, internetin **telefon rehberi** gibidir. Görevi, insanların kolayca hatırlayabileceği **google.com** gibi alan adlarını, bilgisayarların anlayabileceği **IP adreslerine** çevirmektir.

Domain (isim) → **IP adresi** (sunucu adresi)

Kullanıcı tarayıcıya bir adres yazdığında cihaz DNS'e şu soruyu sorar:

“Bu ismin IP adresi nedir?”

DNS de doğru IP adresini döndürür ve bağlantı başlatılır.

DNS'in Mobilde Önemi

Mobil cihazlarda ilk gecikmenin büyük kısmı, DNS çözümlemesinden kaynaklanır:

- Ağ değişince DNS cache sıfırlanır.
- Hücre (cell tower) değişimleri yeni DNS çözümü gerektirir.
- Captive network (otel WiFi giriş ekranı) araya girer.
- Mobil ağlar DNS sorgularını daha sık tekrarlamak zorunda kalır.

Bu nedenle DNS çözümü çoğu zaman ilk API çağrısını yavaşlatan birincil sebeptir.

2. IP Adresi

IP adresi, internet üzerindeki her cihazın **ev adresidir**. Nasıl bir evin fiziksel adresi varsa, internetteki her sunucu ve cihazın da benzersiz bir IP adresi vardır.

IP adresi = Cihazın internetteki adresi

Örneğin:

- 142.250.187.206
- 13.33.12.118

Tarayıcı veya mobil uygulama, DNS'ten aldığı IP adresine bağlanarak sunucuyla iletişime geçer.

Özet

- **DNS** = Alan adını IP adresine çeviren sistem.
- **IP adresi** = İnternetteki cihazların benzersiz adresi.
- **DNS çözümü**, mobil uygulamalarda ilk gecikmenin en büyük sebeplerinden biridir.

Mobil Ağlarda TCP Flow Control, Congestion Control ve Slow Start

Mobil uygulamalarda API çağrılarının bazen yavaşlamasının temel sebebi yalnızca kötü sinyal veya kötü cihaz performansı değildir. Arka planda TCP'nin kendi güvenlik ve denge mekanizmaları çalışır.

Bu bölümde mobil geliştiricilerin bilmesi gereken üç önemli TCP mekanizması sade şekilde açıklanmaktadır:

1. **Flow Control** (alıcıyı korur)
2. **Congestion Control** (ağı korur)
3. **Slow Start** (bağlantıyı yavaşça hızlandırır)

1. Flow Control (Akış Kontrolü)

Flow control, gönderici ile alıcı arasındaki hız farkını dengeleyen mekanizmadır.

Amaç: Gönderici çok hızlı gönderip alıcıyı boğmasın.

Alıcı taraf (örneğin mobil cihaz) kendi buffer kapasitesine göre şu işareti gönderir:

“Benim şu kadar boş yerim var, şimdilik en fazla bu kadar veri gönder.”

Mobilde neden önemli?

- Zayıf sinyalde cihaz paketleri gecikmeli işler.
- Cihaz arka planda yoğun işlem yaparken (kamera, video, oyun) buffer küçülür.
- Flow control devreye girer → veri akışı yavaşlar.

2. Congestion Control (Ağ Tıkanıklığı Kontrolü)

Bu mekanizma **ağ tıkanıklığını** önlemek için kullanılır.

Amaç: Aşırı veri gönderildiğinde ağ çökmemesi için gönderme hızını otomatik azaltmak.

Mobilde bu neden olur?

- Aynı baz istasyonuna çok kullanıcı bağlandığında
- Paket kaybı olduğunda
- Baz istasyonu değişiminde (handover)

Paket kaybı tespit edilirse TCP şöyle der:

“Ağ tıkalı olabilir. Hızı azaltıyorum.”

Bu, mobilde hissedilen “ani yavaşlamaların” ana sebebidir.

3. Slow Start (Yavaş Başlangıç)

Slow Start, bir TCP bağlantısının başlangıçta küçük bir hızla başlaması ve ağ ortamını test ederek yavaş yavaş hızlanmasıdır.

Amaç: Yeni bağlantının ağı tıkamadan kontrollü şekilde hızlanması.

Nasıl çalışır?

1. Başta çok az veri gönderilir.
2. Sunucudan “ACK” geldikçe gönderim miktarı **iki katına çıkar**.
3. Paket kaybı olursa hız azaltılır.

Mobil etkisi:

- İlk API çağrısı her zaman daha yavaştır.
- Büyük dosya indirmeleri ilk birkaç saniyede yavaş başlar.
- Zayıf sinyalde Slow Start sürekli tetiklenebilir.

Gerçek Örnek: Neden API Arada Bir 2 Saniye Gecikiyor?

Bir kullanıcı metroya biniyor:

- Sinyal zayıflıyor → paket kayıpları başlıyor
- TCP bunu tespit ediyor → hızı düşürüyor (Congestion Control)
- Birkaç paket gidip gelince tekrar hızlanıyor (Slow Start)

Kullanıcı şöyle hisseder:

“Bazen API yavaşlıyor, sonra düzeliyor.”

Bu tamamen TCP’nin doğal davranışdır.

Özet

- **Flow Control:** Alıcı çok doluysa hız düşer.
- **Congestion Control:** Ağ tıkalı görünüyorsa hız düşer.
- **Slow Start:** Bağlantı yavaş başlar, sonra hızlanır.

Mobil ağ kararsız olduğu için TCP mekanizmaları sık sık devreye girer. Bu, mobil API çağrılarının zaman zaman dalgalı hissettirmesinin temel sebebidir.

Mobil Ağlarda Wi-Fi ↔ LTE Geçişi, IP Değişimi ve Captive Network Davranışı

Mobil cihazların ağ bağlantısı sabit değildir; kullanıcı hareket ederken cihaz sık sık ağ değiştirir. Bu ağ değişimleri API çağrılarının kesilmesine, gecikmesine veya tamamen başarısız olmasına neden olabilir.

Bu bölümde üç önemli mobil ağ davranışı açıklanmaktadır:

1. Wi-Fi ↔ LTE geçişleri (network switching)
2. IP adresi değişimi
3. Captive network (login isteyen Wi-Fi) tespiti

1. Wi-Fi ↔ LTE Geçişi (Network Switching)

Mobil cihaz, şunlar olduğunda ağı otomatik değiştirir:

- Wi-Fi sinyali zayıflayınca LTE'ye geçer.
- Kullanıcı Wi-Fi ağına bağlanınca LTE kapanır.
- Cihaz, erişim noktaları arasında dolaşır (Wi-Fi roaming).

Etkisi:

- Mevcut TCP bağlantısı kopar.
- DNS cache geçersiz hale gelir.
- IP adresi değiştiği için TLS bağlantısı yeniden kurulur.

Sonuç:

Devam eden istekler genelde **timeout** veya **connection lost** hatası verir.

2. IP Adresi Değişimi

Mobil cihaz ağ değiştirdiğinde IP adresi değişir.

Örnek:

- Wi-Fi'da: 192.168.x.x
- LTE'de: 10.x.x.x veya operatörün NAT IP'si

Her ağ değişiminde:

- IP değişir
- TCP bağlantısı invalid olur
- TLS handshake yeniden yapılır

Bu nedenle aynı API isteği bazen bir anda “network error” verebilir.

3. Captive Network (Otel / Kafe Wi-Fi Login Ekranı)

Captive network, internete çıkmadan önce **login sayfasına yönlendiren** Wi-Fi ağlarıdır.

Örnek:

- Havalimanı
- Otel
- Kafe
- AVM

Bu ağlarda:

- DNS genelde sahte bir IP adresine yönlendirir.
- Tüm HTTP istekleri login sayfasına gider.
- HTTPS istekleri sertifika hatası ile düşer.

Mobil uygulama açısından:

Her şey doğru olsa bile API çağrıları başarısız görünür.

4. Mobil Uygulamalarda Bu Sorunlar Nasıl Algılanır?

Android:

- ConnectivityManager ile ağ değişimi bildirimi alınır.
- VPN / captive portal bilgisi okunabilir.

iOS:

- NWPathMonitor ile network path değişimi izlenir.
- Captive network bilgisi “unsatisfied” durumunda tespit edilir.

5. Bu Sorunlara Karşı Doğru Yaklaşım

- İstekler IP değişiminde otomatik retry edilmelidir.
- Uzun süreli socket/WebSocket bağlantıları yeniden kurulmalıdır.
- DNS çözümlemesi ağ değişimlerinde yeniden yapılmalıdır.
- “No internet” ekranı için gerçek internet testi yapılmalıdır (sadece Wi-Fi bağlı diye internet var sanılmamalı).

Özet

- **Ağ değişimi** = mevcut tüm TCP bağlantıları kopar.
- **IP değişimi** = TLS bağlantısı yeniden kurulur.
- **Captive network** = API istekleri login sayfasına yönlendirilir.

Mobil cihazlarda ağ sabit değildir. Kullanıcı hareket ettikçe bağlantı türü, IP adresi ve internet erişimi sürekli değişir. Bu değişimleri doğru yönetmek, hızlı ve kararlı mobil uygulama geliştirmenin temel şartıdır.

Mobil Uygulamalarda WebSocket, Heartbeat, Keep-Alive ve Reconnect Mantığı

Gerçek zamanlı iletişim gereken mobil uygulamalarda (chat, canlı destek, oyun, WebRTC sinyalleme) WebSocket bağlantıları kullanılır. Ancak mobil ağlar sabit olmadığından WebSocket bağlantıları sık sık kopar ve yeniden kurulması gerekir.

Bu bölümde WebSocket’in mobilde nasıl davranması gerektiği sade şekilde açıklanır.

1. WebSocket Nedir?

WebSocket, istemci ile sunucu arasında **sürekli açık bir bağlantı** kurulmasını sağlayan bir protokoldür.

- HTTP’den farklı olarak çift yönlüdür.
- Bağlantı açık kaldığı sürece anlık veri gönderilebilir.
- Chat ve bildirim sistemlerinin temelidir.

2. Mobilde WebSocket Neden Kopar?

Mobil ağlar sabit olmadığından WebSocket bağlantısı birçok nedenle kopabilir:

- Wi-Fi ↔ LTE geçişi
- IP adresi değişimi
- Ekran kapanınca işletim sisteminin bağlantıyı kısıtlaması
- Timeout (uzun süre veri gitmeyince ağ tarafından kapatılır)
- Zayıf sinyal veya paket kaybı

WebSocket kopması bir hata değildir; mobilde **beklenen normal davranıştır**.

3. Heartbeat Nedir?

Heartbeat (ping/pong), bağlantının hala canlı olduğunu kontrol eden küçük paketlerin belli aralıklarla gönderilmesidir.

Amaç: “Bağlantı hâlâ açık mı?” sorusunu periyodik olarak kontrol etmek.

Örnek aralıklar:

- Her 20–30 saniyede bir “ping”
- Sunucudan “pong” cevabı beklenir

Mobilde neden şart?

Çünkü:

- Ağ router'ları uzun süre sessiz kalan bağlantıları kapatır.
- LTE ağları inaktif TCP bağlantılarını düşürür.

4. Keep-Alive Nedir?

Keep-alive, işletim sistemi veya HTTP/TCP seviyesindeki mekanizmaların bağlantıyı açık tutmaya çalışmasıdır.

- TCP keep-alive paketleri
- Android / iOS OS-level mekanizmaları

Ancak mobilde OS uygulamaları arka planda agresif şekilde durdurabileceği için **yalnızca keep-alive'a güvenilemez**. Heartbeat bu yüzden zorunludur.

5. Reconnect Mantığı (Yeniden Bağlanma)

Mobil WebSocket yapısının en kritik kısmı **doğru reconnect algoritmasıdır**.

Reconnect şu durumlarda tetiklenmelidir:

- Heartbeat cevabı alınamadığında
- IP adresi değiştiğinde
- Ağ tamamen kesildiğinde
- Sunucu “close” mesajı gönderdiğinde

Reconnect algoritması çoğunlukla **exponential backoff** ile yapılır:

1. deneme → hemen
2. deneme → 1 saniye sonra
3. deneme → 2 saniye sonra
4. deneme → 4 saniye sonra
- ...

6. Mobilde WebSocket İçin En İyi Pratikler

- Heartbeat zorunlu (ping/pong)
- Otomatik reconnect + exponential backoff
- Ağ değişimi algılandığında bağlantıyı yeniden kurmak
- “No internet” durumunda beklemek, zorla bağlanmaya çalışmamak
- Arka planda bağlantının kapanmasını OS seviyesinde kabul etmek

Özet

- **WebSocket:** Gerçek zamanlı sürekli bağlantı.
- **Heartbeat:** Bağlantının canlı olup olmadığını kontrol eder.
- **Keep-alive:** Bağlantının kapanmaması için düşük seviye sinyaller.
- **Reconnect:** Kopma durumunda kontrollü şekilde yeniden bağlanma.

Mobilde WebSocket bağlantısı her zaman kopar. Önemli olan kopmayı engellemek değil, doğru ve hızlı bir şekilde **yeniden bağlanabilmektir**.

Mobil Uygulamalarda Cache: Memory Cache, Disk Cache ve Stale-While-Revalidate

Mobil uygulamalarda ağ gecikmesi, paket kaybı ve enerji tüketimi yüksek olduğundan, doğru cache stratejileri hem performansı hem de kullanıcı deneyimini önemli ölçüde iyileştirir. Bu bölümde mobil geliştiricilerin bilmesi gereken üç temel cache yaklaşımı açıklanır:

1. Memory Cache
2. Disk Cache
3. Stale-While-Revalidate (SWR)

1. Memory Cache (Bellek Üstü Cache)

Memory cache, verilerin doğrudan RAM üzerinde tutulmasıdır.

- En hızlı cache türüdür.
- Uygulama kapatılınca kaybolur.
- Tekrar eden ekran geçişlerinde veriyi anında gösterir.

Mobilde kullanım örnekleri:

- Profil bilgisi
- Ayarlar
- UI listeleri (kısa ömürlü veriler)
- Görsel placeholder cache

Memory cache = “Bu veriyi çok hızlı lazım olacak, RAM’de tutayım.”

2. Disk Cache (Kalıcılık İçin Cache)

Disk cache, verilerin cihazın depolama alanına kaydedilmesidir.

- Memory cache’ten yavaştır.
- Uygulama kapansa da veri kalır.
- Offline-first uygulamalar için zorunludur.

Mobilde kullanım örnekleri:

- Haber listeleri

- E-ticaret ürün listesi
- Kullanıcının geçmiş mesajları
- Büyük JSON yanıtları

Disk cache genellikle şu stratejiyle çalışır:

- Eğer network varsa: veriyi yenile
- Network yoksa: disk cache'ten oku

3. Stale-While-Revalidate (SWR) Stratejisi

SWR, modern mobil uygulamalarda en çok kullanılan cache modelidir.

“Önce eski veriyi hemen göster, sonra arka planda yenisini al.”

Adımlar:

1. Disk veya memory cache'te veri varsa **hemen UI'a verilir**.
2. Arka planda sunucuya yeni istek gönderilir.
3. Yeni veri gelince UI güncellenir ve disk/memory cache yenilenir.

Neden SWR Mobilde Kritik?

- Latency yüksek olsa bile UI **anında açılır**.
- Network kötü ise bile uygulama **offline gibi çalışır**.
- Battery cost düşer (daha az gereksiz API çağrısı).
- Kullanıcı anlık veri görür; “loading” ekranları azalır.

SWR = “Hızlı görünen ama her zaman güncel kalan UI.”

4. Mobilde Cache Süreleri

Önerilen TTL değerleri:

- Kritik veriler → 30 saniye
- Listeler / feed'ler → 1 dakika
- Görseller → 1 gün

5. En İyi Pratikler

- Sık kullanılan ekranlar için memory cache zorunlu.
- Büyük listeler disk üzerinde tutulmalı.
- API yanıtı ilk önce memory cache'e, sonra disk'e yazılmalı.
- SWR modeli ile “instant UI” sağlanmalı.
- Cache invalidation açıkça belirlenmeli.

Özet

- **Memory Cache:** En hızlı, geçici veri tutar.
- **Disk Cache:** Kalıcıdır, offline kullanım için gereklidir.
- **SWR:** Eski veriyi hemen gösterip arka planda yeniler.

Mobil uygulamalarda cache stratejisi, performansı belirleyen en önemli mimari kararlardan biridir. Cache doğru kullanıldığında uygulama hızlanır, daha az data kullanır ve daha az enerji tüketir.

CDN, Edge ve Geo Routing: Mobil Kullanıcı Neden Farklı Sunuculara Yönlendirilir?

Mobil kullanıcılar dünyanın farklı bölgelerinde bulunduğu için aynı API çağrısı veya aynı görsel isteği farklı coğrafi sunuculara yönlendirilebilir. Bu yönlendirme performansı artırmak için yapılır.

Bu bölümde mobil geliştiricilerin bilmesi gereken üç temel kavram açıklanmaktadır:

1. CDN (Content Delivery Network)
2. Edge Sunucuları
3. Geo Routing (Coğrafi Yönlendirme)

1. CDN (Content Delivery Network) Nedir?

CDN, dünyanın farklı noktalarına dağıtılmış sunucu ağlarıdır. Amaç, kullanıcıya en yakın sunucudan içerik göndermektir.

- Görseller
- JSON dosyaları
- Statik içerikler
- Videolar

CDN üzerinden gelir.

Avantajları:

- Daha düşük gecikme (latency)
- Daha hızlı DNS yanıtı
- Daha az paket kaybı
- Sunucu yükünün azalması

CDN = İçeriğin kullanıcının cebine en yakın noktaya kopyalanması.

2. Edge Sunucuları

Edge, CDN ağındaki “kullanıcıya en yakın fiziksel sunucudur”.

Örnek:

- İstanbul’daki kullanıcı → İstanbul edge sunucusu
- Berlin’deki kullanıcı → Frankfurt edge sunucusu

Mobilde edge neden önemli?

- LTE veri yolunda gecikme zaten yüksektir, edge bu gecikmeyi azaltır.
- Uzun mesafeli fiber geçişlerini engeller.
- Büyük dosyalar daha hızlı yüklenir.

3. Geo Routing (Coğrafi Yönlendirme)

Geo routing, kullanıcının IP adresine göre en yakın edge veya backend sunucusuna yönlendirilmesidir.

Yönlendirme şu faktörlere bakarak yapılır:

- Kullanıcının coğrafi konumu (IP bazlı)
- En yakın edge sunucusunun yoğunluğu
- Sunucular arası gecikme ölçümleri

Örnek:

Türkiye’deki bir kullanıcı:

- api.example.com → İstanbul edge
- images.example.com → İstanbul edge üzerinde cache

Aynı uygulamayı Almanya’da açan kullanıcı:

- api.example.com → Frankfurt edge
- images.example.com → Frankfurt edge cache

Aynı domain, farklı ülkelere farklı sunucular üzerinden hizmet verir.

4. Mobil Uygulamalar İçin Neden Kritik?

- Mobil ağlarda sinyal değişkendir; gecikme zaten yüksektir.
- CDN/edge latency’i büyük ölçüde azaltır.
- Görseller daha hızlı yüklenir.
- API cevapları daha kararlı hale gelir.
- Kullanıcı hareket ederken (örneğin şehir içinde) en yakın edge otomatik değişir.

5. CDN Kullanan Mobil Uygulama Pratikleri

- Görseller mutlaka CDN üzerinden gelmelidir.
- API cevapları mümkünse edge üzerinde cache’lenmelidir.
- Mobil cihaz için “Accept-Encoding: gzip/br” etkin olmalıdır.
- CDN tarafında “stale-while-revalidate” tanımlanmalıdır.

Özet

- **CDN:** İçeriği dünya geneline dağıtır.
- **Edge:** Kullanıcıya en yakın sunucu.
- **Geo Routing:** Kullanıcının IP'sine göre en hızlı node'a yönlendirme.

Mobil kullanıcı dünyanın neresinde olursa olsun içeriğin ve API'nın hızlı olması, doğru CDN, edge ve geo routing stratejileriyle mümkündür.

HTTP Keep-Alive ve Short-Lived Connections: Mobilde Hangisi Daha Verimli?

Mobil cihazların ağ bağlantısı sabit olmadığı için HTTP isteklerinin bağlantı yönetimi performansı doğrudan etkiler. Bu bölümde iki temel bağlantı stratejisi açıklanmaktadır:

1. **Keep-Alive** (Persistent Connection)
2. **Short-Lived** (Her istekte yeni bağlantı)

1. Short-Lived Connection Nedir?

Short-lived bağlantı, her API isteği için:

1. DNS çözümü
2. TCP bağlantısı
3. TLS handshake

işlemlerinin **tekrar** yapılmasıdır.

Dezavantajları:

- Her istek 200–800 ms ekstra gecikme alır.
- Battery cost çok yükselir.
- Mobil ağ değişimlerinde daha fazla hata oluşur.

Short-lived = “Her istekte sıfırdan telefon edip konuşmaya başlamak.”

2. Keep-Alive (Persistent Connection) Nedir?

Keep-alive, bir TCP + TLS bağlantısını bir kez açıp **birden fazla istekte tekrar kullanmak** anlamına gelir.

Bu sayede:

- DNS çözümü yalnızca ilk istekte yapılır.
- TCP bağlantısı korunur.
- TLS handshake tekrar yapılmaz.

Avantajları:

- İkinci ve sonraki istekler 5–20 ms gibi çok hızlıdır.
- Battery tüketimi belirgin şekilde düşer.
- Sunucuya daha az yük gider.

Keep-alive = “Bağlantıyı açık tut, tekrar tekrar kullan.”

3. Mobil Cihazlarda Keep-Alive Neden Daha Önemlidir?

Mobil ağlar sabittir:

- LTE sinyali sürekli değişir
- Baz istasyonları arasına geçilir
- Wi-Fi ↔ LTE geçişleri sık yaşanır

Bu nedenle short-lived bağlantılar:

- sık sık zaman aşımına uğrar
- TCP yeniden kurulur
- uygulama **donuyor gibi hissedilir**

Keep-alive sayesinde:

- sabit bir bağlantı sürekli yeniden kullanılabilir
- mobil ağ dalgalanmaları daha az hissedilir
- API performansı çok daha stabil olur

4. Zayıf Sinyalde Neden Short-Lived Çok Kötüdür?

Her yeni bağlantı açmak:

- 3 kez paket alışverişi (TCP handshake)
- 2 kez paket alışverişi (TLS handshake)

gerektirir.

Zayıf LTE sinyalinde bu işlemler saniyelere yayılabilir.

Bu durumda keep-alive her zaman daha hızlıdır.

5. Modern Mobil Kütüphaneler Keep-Alive Nasıl Uygular?

React Native:

- Fetch API → otomatik keep-alive
- Axios → keep-alive adapter ile gelişmiş kullanım
- React Query → bağlantı reuse edilir

Android (OkHttp):

- Keep-alive varsayılan olarak etkindir.
- Connection pool ile birden çok bağlantı reuse edilir.

iOS (NSURLSession):

- HTTP/2 keep-alive otomatik çalışır.
- Aynı host için bağlantı paylaşılır.

6. Keep-Alive’ın Riskleri

- Bağlantı uzun süre açık kalınca “stale” hale gelebilir.
- Bazı load balancing mimarileri kısa timeout kullanır.
- Ağ değişimi olursa bağlantı kopar.

Bu nedenle:

- Heartbeat (ping) önerilir.
- Bağlantı koparsa otomatik retry yapılmalıdır.

Özet

- **Short-Lived:** Her istekte bağlantı sıfırdan kurulur → yavaş + pahalı.
- **Keep-Alive:** Bağlantı tekrar kullanılır → hızlı + enerji tasarruflu.

Mobil uygulamalarda performans ve enerji verimliliği için **Keep-Alive** neredeyse her zaman en doğru bağlantı yönetimi stratejisidir.

Mobil Ağlarda TCP ve UDP: Hangisi Ne Zaman Kullanılır?

Mobil cihazlarda ağ koşulları sürekli değiştiği için (sinyal, hız, paket kaybı), doğru protokol seçimi uygulamanın kararlılığı ve performansı açısından çok önemlidir.

Bu bölümde TCP ve UDP'nin mobil ortamdaki davranışları ve hangi durumda hangisinin tercih edilmesi gerektiği sade şekilde açıklanır.

1. TCP Nedir?

TCP, **güvenilir** ve **sıralı** veri iletimi sağlar.

Temel özellikleri:

- Paketler eksiksiz ulaşır.
- Paketler sırayla gelir.
- Kaybolan paketler yeniden gönderilir (retransmission).
- Bağlantı kurulması gerekir (handshake).

Bu nedenle:

TCP = “Doğruluk önemliyse, hız ikinci planda kalabilir.”

Mobilde kullanım alanları:

- API çağrıları
- HTTPS bağlantıları
- Dosya indirme / yükleme
- Kullanıcı verisi (mesaj, profil)

2. UDP Nedir?

UDP, **hızlı** fakat **güvenilir olmayan** bir protokoldür.

Özellikleri:

- Bağlantı kurulmaz (connectionless).
- Kaybolan paket tekrar gönderilmez.
- Paket sırası garanti edilmez.

Bu nedenle:

UDP = “Hız çok önemliyse, bazen paket kaybına tolerans vardır.”

Mobilde kullanım alanları:

- WebRTC toplantıları
- Canlı video yayını
- Sesli görüşme
- Online oyunlar
- Real-time çizim / konum senkronizasyonu

3. Mobilde TCP’nin Dezavantajları

Mobil ağlar kararsızdır ve TCP şu durumlarda ciddi yavaşlamalar yaratır:

- Paket kaybı olunca hız düşer (congestion control).
- Handover (baz istasyonu geçişi) olunca bağlantı kopar.
- LTE ↔ Wi-Fi değişiminde bağlantı tamamen resetlenir.

Bu nedenle:

TCP, sık sinyal değişen mobil ortamlarda “dalgalı” performans gösterir.

4. Mobilde UDP’nin Avantajları

UDP paket kaybı olduğunda durmaz, hızını düşürmez. Bu nedenle gerçek zamanlı uygulamalarda idealdir.

Özellikle WebRTC şu nedenle UDP kullanır:

- Ses/video için “biraz paket kaybı” kaliteyi bozmaz.
- Ama gecikme artarsa konuşma tamamen bozulur.

Gerçek zamanlı iletişimde gecikme, doğruluktan daha kritiktir.

5. WebRTC Neden UDP Kullanır?

WebRTC, mobilde kararsız ağları tolere edebilmek için UDP üzerine **kendi hata düzeltme mekanizmalarını** ekler:

- Jitter buffer
- FEC (Forward Error Correction)
- Packet prioritization

Böylece:

- Gecikme çok düşüktür
- Paket kaybı ses/video kalitesini tamamen bozmaz

6. Hangi Durumda TCP, Hangi Durumda UDP?

TCP kullan:

- Doğruluk önemliyse
- Kaybolan paket kabul edilemezse
- Veri mutlaka eksiksiz gelmeli ise
- API çağrıları, dosya transferleri, ödeme işlemleri

UDP kullan:

- Hız kritikse
- Birkaç paketin kaybolması sorun değilse
- Ses/video/chat real-time akıyorsa
- Oyun ve canlı yayın gibi durumlarda

Özet

- **TCP:** Güvenilir ama yavaş — API ve kritik veri için idealdir.
- **UDP:** Çok hızlı ama güvenilir değil — WebRTC ve real-time iletişim için idealdir.

Mobil ağda en önemli karar: “Doğruluk mı önemli, hız mı?” Cevap
hıssa UDP, doğruluksa TCP seçilir.

Mobilde DNS Caching (iOS / Android)

Mobil uygulamalarda bir domain'e yapılan ilk istek genellikle yavaştır. Bunun temel sebebi DNS çözümlemesinin (DNS lookup) tekrar yapılmasıdır. Mobil işletim sistemlerinde DNS cache masaüstüne göre daha agresif biçimde temizlenir.

1. Android DNS Davranışı

- Android, DNS sonuçlarını **kısa süreli** cache'ler (genelde 30–60 sn).
- Wi-Fi ↔ LTE geçişinde DNS cache **tamamen temizlenir**.
- Her ağ değişiminde domain yeniden çözülür.

2. iOS DNS Davranışı

- iOS, DNS'i connection-level cache olarak tutar.
- Uygulama arka plana gidip geri geldiğinde cache çoğunlukla silinir.
- Ağ geçişinde DNS çözümü **yeniden yapılır**.

3. Sonuç (Performansa Etkisi)

- İlk API çağrısı her zaman daha yavaştır (DNS yüzünden).
- Ağ değişimlerinde timeout oluşmasının ana sebebi DNS'tir.
- Bu yüzden CDN, kısa DNS zinciri ve kalıcı IP yönlendirmesi önerilir.

Mobil cihazlarda DNS, masaüstüne göre daha sık temizlendiği için **ilk çağrı yavaş**, ağ geçişleri ise **hata üretmeye daha yatkındır**.

Authorization: JWT, OAuth2 ve Token Rotation (Mobil Özet)

Mobil uygulamalarda oturum yönetimi genellikle iki ana kavram üzerine kurulur: **access token** ve **refresh token**. Bu yapı, güvenliği artırmak ve mobil cihazlardaki ağ değişikliklerine rağmen oturumu korumak için kullanılır.

1. Access Token

- Kısa ömürlüdür (5–15 dakika).
- API isteklerinde **Authorization: Bearer <token>** olarak gönderilir.
- Çalınırsa uzun süre kullanılamaz (çünkü süresi kısadır).

Access token = “Hızlı tükenen giriş bileti.”

2. Refresh Token

- Çok daha uzun ömürlüdür (günler/haftalar).
- Sunucuya gönderildiğinde yeni bir access token üretir.
- Güvenlik nedeniyle genellikle sadece **login sunucusuna** gönderilir.

Refresh token = “Yeni erişim bileti üretme hakkı.”

3. Token Rotation (Yenileme Döngüsü)

Token rotation, her refresh işlemi sonrasında:

1. yeni bir access token,
2. **ve yeni bir refresh token**

üretme prensibidir.

Amaç:

- çalınmış refresh token’ların kullanılmasını zorlaştırmak,
- oturumu sürekli güncel ve güvenli tutmak.

4. Mobilde Neden Gereklidir?

- Mobil cihazlar sık sık IP değiştirir.
- Ağ kopmaları sonucu access token süreleri dolabilir.
- Kullanıcıları sürekli yeniden login yaptırmamak gerekir.

Bu nedenle API çağrılarını şu döngüyle çalışır:

1. Access token API'ya gönderilir.
2. Eğer 401 alınır → refresh token ile yenileme yapılır.
3. Yeni token'larla API isteği tekrar edilir.

5. Güvenlik Açısından Önemli Noktalar

- Access token kısa ömürlü olmalı.
- Refresh token **hiçbir zaman üçüncü API servislerine gönderilmemeli.**
- Refresh token çalınırsa rotation sayesinde hızla geçersiz olur.
- Mobilde token'lar **secure storage** içinde tutulmalıdır:
 - iOS: Keychain
 - Android: Keystore
 - React Native: Secure Storage / Keychain

Özet

- **Access token:** Kısa ömürlü izin bileti.
- **Refresh token:** Access token yenileyici uzun ömürlü anahtar.
- **Token rotation:** Her yenilemede hem access hem refresh token'ın değişmesi.

Mobil uygulamalarda doğru authorization yapısı: **kısa ömürlü access token + güvenli refresh token + otomatik token rotation** kombinasyonudur.

API Gateway ve Rate Limiting (Mobilde Etkisi)

API Gateway, mobil uygulamanın tüm API çağrılarının geçtiği merkezi kontrol katmanıdır. Mobilde yaşanan birçok gizli performans probleminin arkasında gateway politikaları bulunur.

1. API Gateway Nedir?

- Tüm API trafiğinin giriş kapısıdır.
- Auth, rate limit, routing ve güvenlik kontrolleri burada yapılır.
- Mobil cihaz backend'e doğrudan konuşmaz; gateway'e konuşur.

Gateway = “Backend dünyasının gümrük kapısı.”

2. Mobilde Neden Kritik?

Mobil cihazlarda bağlantı sık sık kopup bağlandığı için:

- aynı istek birkaç kez tekrar edilebilir,
- hızlı ardışık isteklerde rate limit'e takılabilir,
- zayıf ağda threshold'lar hızlıca dolabilir.

Bu yüzden gateway politikaları mobil uygulamada beklenmedik hatalara yol açabilir.

3. Rate Limiting Türleri

- **IP-based:** Mobilde problemli, çünkü mobil operatör NAT kullanır.
- **User-based:** Kimlik doğrulaması olan uygulamalar için en yaygını.
- **Endpoint-based:** Belirli API çağrılarının saniye başına limiti.

Mobil kullanıcılar aynı baz istasyonu IP'sini paylaştıkları için **IP-based rate limit en sık hata yaratan tiptir.**

4. Mobilde Rate Limit Nasıl Hissedilir?

- Çok hızlı scroll yapınca API'nın “bir anda durması”
- Infinite list'in aniden kesilmesi
- “Unexpected 429 too many requests” hatası
- Birkaç refresh sonrası API'nın cevap vermemesi

Kısacası: “Uygulama bozuk gibi davranır.”

5. Gateway’in Mobilde Yaptığı Diğer İşler

- Token doğrulama (401)
- CORS kontrolü
- Request body size limit
- Timeout ve retry politikaları
- Routing (canary release, blue-green)

Bu politikaların herhangi biri mobilde yavaşlık veya hata yaratabilir.

6. Mobil Geliştiricinin Dikkat Etmesi Gerekenler

- Aynı API’yı kısa sürede çok kez çağırma (debounce kullan).
- Retry yaparken exponential backoff kullan.
- 429 gelirse otomatik yeniden deneme **yapma** → bekle.
- Pagination’da gereksiz paralel istek gönderme.

Özet

- API Gateway, tüm mobil trafiğin kontrol merkezi.
- Rate limit, zayıf veya çok hızlı kullanıcı etkileşimlerinde kolay tetiklenir.
- 429 hataları çoğunlukla backend değil gateway kaynaklıdır.
- Mobil uygulamada debounce, retry ve caching gateway yükünü azaltır.

Mobil uygulamada “ani kesilmeler”, “scroll’un durması” veya “beklenmedik 401/429 hataları” çoğu durumda backend değil, **gateway politikaları yüzünden olur**.

Download / Upload Optimizasyonu: Chunking, Resume ve Range Requests

Mobil ağılar stabil olmadığı için büyük dosya işlemleri sık sık yarıda kesilir. Bu nedenle modern mobil uygulamalar tek parça yükleme yerine **parçalı (chunked)** aktarım kullanır.

1. Chunked Upload (Parçalı Yükleme)

- Büyük dosya, küçük parçalara (ör. 1 MB) bölünür.
- Her parça ayrı bir API isteği ile gönderilir.
- Bağlantı koparsa sadece eksik parça tekrar yüklenir.

Avantajları:

- Zayıf ağda yükleme daha stabil olur.
- Yarıda kesilse bile ilerleme kaybolmaz.

2. Resume Upload

Yükleme kesildiğinde kaldığı yerden devam edebilmek için:

1. Sunucu hangi chunk'ların alındığını tutar.
2. Mobil cihaz sadece eksik parçaları tekrar gönderir.

Resume = “Baştan değil, kaldığın yerden devam et.”

3. Range Requests (Bölmeli İndirme)

Büyük dosyalar için indirme şu şekilde yapılır:

- İstemci **Range**: `bytes=0-999999` gibi bir aralık ister.
- Sunucu sadece istenen bölümü döner.
- İndirme yarıda kesilirse yeni range ile devam edilir.

Bu mekanizma özellikle video indirmede kullanılır.

4. Mobil İçin Neden Zorunludur?

Mobil ağlarda sık görülen senaryolar:

- LTE ↔ Wi-Fi geçişi
- Sinyal dalgalanması
- Paket kaybı
- Timeout sonrası yeniden deneme

Tek parça upload/indirme bu durumda çoğunlukla başarısız olur.

5. Kullanılan Teknolojiler / Kütüphaneler

- **S3 Multipart Upload** (en yaygın çözüm)
- **Tus Protocol** (resume upload için özel standart)
- **HTTP Range Requests** (sunucu tarafında destek gerekir)
- **React Native için:**
 - react-native-background-upload
 - expo-file-system (resume destekli)

Özet

- Tek parça upload/indirme mobilde güvenilir değildir.
- Parçalı (chunked) aktarım her zaman daha hızlı ve stabildir.
- Resume mekanizması kopmalarda zaman kazandırır.
- Range Requests büyük dosya indirmede zorunludur.

Mobilde büyük dosya işlemleri yapacaksan çözüm bellidir: **Chunk + Resume + Range**.

Offline-First ve Network Reachability (Mobil En İyi Uygulamalar)

Mobil cihazlar sürekli çevrimdışı ve çevrimiçi durumlar arasında geçiş yapar. Bu nedenle modern mobil uygulamalar “offline-first” yaklaşımını benimser: bağlantı yokken uygulama çalışmaya devam eder, bağlantı geldiğinde senkronizasyon yapılır.

1. Offline-First Nedir?

- Kullanıcı verisi önce lokal depoya yazılır (cache, MMKV, SQLite).
- Ağ gelince arka planda sunucuya gönderilir (sync queue).
- Kullanıcı bağlantısızken bile uygulama sorunsuz ilerler.

Offline-first = “Bağlantıya güvenme, veri her zaman ulaşılabilir olsun.”

2. Network Reachability Nedir?

Reachability, cihazın gerçek ağ durumunu izleyen mekanizmadır:

- Online → API istekleri aktif
- Offline → tüm istekler kuyruklanır veya önbellekten okunur
- Değişim → retry tetiklenir

React Native:

- @react-native-community/netinfo

iOS:

- NWPathMonitor

Android:

- ConnectivityManager

3. Offline-First İçin Temel Teknikler

- **Local cache:** MMKV, SQLite, WatermelonDB, Room
- **Write queue:** istekler sıraya alınır, ağ gelince gönderilir
- **Mutation deduplication:** aynı veri birden fazla kez gönderilmez
- **Conflict resolution:** hem lokal hem sunucu değişmişse birleştirme yapılır

4. Reachability'nin Mobil İçin Zor Yanları

- LTE sinyali varken internet olmayabilir (captive network)
- Wi-Fi aktif olsa da router interneti olmayabilir
- Sinyal anlık dalgalanabilir (flaky network)

Bu yüzden:

- API başarısız olunca **hemen offline moduna geçilmez**
- birkaç başarısız istekten sonra offline kabul edilir

5. En İyi Retry Stratejisi

- Exponential backoff ($1s \rightarrow 2s \rightarrow 4s \rightarrow 8s$)
- Jitter (rastgele küçük gecikme ekleme)
- Sabit tekrar sayısı (ör. max 5 retry)

Amaç:

- Gateway'i boğmamak
- Battery drain yaratmamak
- Ağ gereksiz tüketmemek

Özet

- Mobil uygulama her zaman offline çalışabilmelidir.
- Network reachability $\rightarrow$ ağ dalgalanmalarını doğru algılar.
- Retry mekanizması akıllı olmalı (backoff + jitter).
- Veri önce lokal depoya yazılır, sonra senkronize edilir.

Gerçek mobil dayanıklılığı, “internet yokken bile çalışan uygulama” tasarlamakla başlar.

Offline-First Mutations: TanStack Query ile “Önce Lokal DB, Sonra Sunucu” Modeli

Mobil cihazlarda ağ sürekli kesildiği için mutasyonların (veri gönderme işlemleri) her zaman sunucuya ulaşması garanti değildir. TanStack Query, “offline-first” çalışma modelini destekleyerek önce verinin lokal depoya kaydedilmesine, ardından bağlantı geldiğinde sunucuya otomatik gönderilmesine izin verir.

1. Temel Prensipler

1. Kullanıcı bir mutasyon başlatır (ör. form gönderimi).
2. Veri **önce lokal depoya** (MMKV, SQLite, WatermelonDB) yazılır.
3. TanStack Query mutasyonu **kuyruğa alır**.
4. İnternet yoksa sunucuya gönderilmez.
5. Bağlantı geri geldiğinde mutasyon otomatik olarak yeniden çalışır.

Offline-first mutation = “Veriyi önce sakla, sonra fırsat bulunca gönder.”

2. TanStack Query Ayarı

Aşağıdaki ayar, bir mutasyonun offline modda kuyruğa alınmasını sağlar:

```
typescript useMutation( mutationFn: sendToServer, retry: true, network-
Mode: 'offlineFirst' );
networkMode='offlineFirst' TanStack Query'e şu talimatı verir:
```

- İnternet yoksa mutasyonu çalıştırma.
- Kuyruğa ekle ve beklet.
- Bağlantı gelince otomatik gönder.

3. Lokal Veri Yazımı

Mutasyon tetiklendiğinde veri önce lokal DB'ye kaydedilir:

```
typescript saveLocal(data); // MMKV veya SQLite mutata(data); // Kuyruğa
alınır
```

Böylece kullanıcı çevrimdışı olsa bile uygulama akışı kesilmez.

4. İnternet Geldiğinde Ne Olur?

TanStack Query:

- İnternet bağlantısını NetInfo / ConnectivityManager ile izler.
- Kuyruktaki tüm offline mutasyonları sırayla çalıştırır.
- Başarılı olanları kaldırır, başarısız olanları tekrar dener.

5. Neden Mobilde Kritik?

- LTE–WiFi geçişlerinde mutasyonlar kaybolmaz.
- Kullanıcı offline modda bile veri girişi yapabilir.
- Uygulama “çevrimdışı çalışan” hissi verir.

Özet

- TanStack Query, **offline mutation queue** mekanizmasını doğal olarak destekler.
- Veri önce lokal depoya yazılır, sonra sunucuya gönderilir.
- İnternet geldiğinde tüm mutasyonlar otomatik olarak yeniden denenir.

TanStack Query + MMKV/SQLite = Gerçek bir “offline-first” mobil mimarisi.

Mobil Ağ Hataları: Server Unreachable, No Internet, Flaky Network

Mobil cihazlar masaüstüne göre çok daha fazla ve tahmin edilemez ağ hatası üretir. Bu hatalar çoğu zaman cihaz çevrimdışıyken değil, **çevrimiçi görünmesine rağmen** yaşanır. Bu bölüm mobilde en yaygın ağ hata türlerini özetler.

1. No Internet (Gerçekte internet yok)

Cihaz bağlantılı görünse bile aşağıdaki durumlarda internet yoktur:

- Wi-Fi bağlı fakat modem internete çıkamıyor.
- LTE sinyali var ama operatör veri aktarımını kesmiş.
- Captive portal (otel / havaalanı Wi-Fi) login gerektiriyor.

Bu durumda API istekleri **hemen timeout** olur.

2. Server Unreachable (Sunucuya hiç ulaşamıyor)

Aşağıdaki durumlarda sunucuya bağlantı kurmak mümkün olmaz:

- DNS çözilemiyor (DNS lookup failure).
- Baz istasyonu değişiminde IP resetleniyor.
- Gateway, yeni bağlantıyı kabul etmiyor.

Belirti:

Cihaz çevrimiçi, fakat uygulama sürekli “network error” veriyor.

3. Flaky Network (Dalgalı ağ)

Flaky network, “var gibi ama stabil değil” anlamına gelir.

Belirtiler:

- Paket kaybı yüksek.
- RTT (latency) sürekli değişiyor.
- Baz istasyonu zayıf sinyal veriyor.

Sonuç:

- WebSocket sık sık kopar.
- Video call kalitesi bozulur.
- API istekleri bazen çok hızlı, bazen çok yavaş olur.

4. Wi-Fi ↔ LTE Geçişi

Mobil cihaz ağ geçişi sırasında:

- IP adresini değiştirir.
- DNS cache temizlenir.
- Tüm TCP bağlantıları kopar.

Bu nedenle:

- Upload / download yarıda kalır.
- WebSocket kapanır.
- İlk API çağrısı çok yavaş olur.

5. Timeout Hatalarının Nedenleri

Timeout genellikle üç nedenle oluşur:

1. DNS çözümü başarısız.
2. TCP bağlantısı kurulamadı.
3. Paket kaybı veya gecikme çok yüksek.

6. Mobil İçin En İyi Korunma Yöntemleri

- **Retry + exponential backoff** kullan.
- **Timeoutları kısa tut**, ilk paket gelmezse hemen bırak.
- **Reachability** ile gerçek ağ durumunu izle.
- **Offline-first** stratejilerle veri kaybını önle.
- **Chunked upload/download** ile kopmalara dayanıklı ol.

Özet

- Mobil ağlar çoğu zaman “bağlı gibi görünüp bağlı olmayan” durumlar yaratır.
- DNS, paket kaybı ve ağ geçişleri mobil hataların ana sebepleridir.
- Dayanıklı bir uygulama için timeout, retry, reachability ve offline-first stratejileri zorunludur.

Gerçek mobil dayanıklılığı, ağ mükemmeliş gibi değil, **ağ kötüymüş gibi** tasarım yaparak elde edilir.